EF Core: AutoDetectChangesEnabled vs. Manual Detection

Visual Lesson Summary • Defaults, bulk updates, safe pattern, and performance

1. Core Idea

AutoDetectChangesEnabled controls whether EF Core automatically performs change detection during operations that normally trigger it. The default is **true**, which is convenient for most CRUD applications.

2. Automatic vs. Manual

Aspect	Automatic (true)	Manual control (false)
How it works	EF Core detects changes during common operations such as SaveChanges().	You explicitly call DetectChanges() when needed.
Benefit	Less code; fewer opportunities to forget detection.	Can reduce repeated scans in some large workloads.
Risk	More automatic checks can add overhead.	Forgetting detection can cause changes not to be recognized as expected.

3. Automatic Mode

```
context.ChangeTracker.AutoDetectChangesEnabled = true;
```

For ordinary code, changing a tracked entity and calling **SaveChanges()** usually works without a manual DetectChanges() call.

4. Manual Mode for Bulk Work

```
context.ChangeTracker.AutoDetectChangesEnabled = false;\n\nvar students = context.Students.ToList();\nf
oreach (var student in students)\n{\n    student.PhoneNumber = "0790000000";\n}\n\ncontext.ChangeTracke
r.DetectChanges();\ncontext.SaveChanges();
```

Manual detection may help when many tracked entities are modified and repeated detection would be costly. Profile first; it is not a universal optimization.

5. Safer Temporary Pattern

```
context.ChangeTracker.AutoDetectChangesEnabled = false;\ntry\n{\n    // Perform bulk modifications\n
 context.ChangeTracker.DetectChanges();\n    context.SaveChanges();\n}\nfinally\n{\n    context.ChangeT
racker.AutoDetectChangesEnabled = true;\n}
```

The **finally** block restores the setting even if an exception occurs. Avoid leaving automatic detection disabled unintentionally.

6. Performance: Check the Real Bottleneck

• Many tracked entities can increase CPU comparisons and memory use.

• Manual detection can reduce repeated scans in some bulk loops.

• It does not automatically fix slow queries, missing indexes, N+1 queries, or loading too much data.

• For read-only queries, consider AsNoTracking() when appropriate.

Common mistakes
• Disabling auto detection before there is evidence of a bottleneck.
• Forgetting DetectChanges() when manual detection is required.
• Leaving the setting disabled beyond the intended operation.
• Assuming manual detection always improves performance.

7. Check the Current Setting

```
bool enabled = context.ChangeTracker.AutoDetectChangesEnabled;
```

8. Mental Anchor

Keep automatic detection enabled by default. Use manual detection only when a large workload and performance evidence justify the extra control.